CATIA_V5_EasyToolbar

V2.01

Technical Documentation & User Manual

Support & Contact: looksail@gmail.com

Copyright © 2026 All Rights Reserved.

CONTENTS

1 Why We Developed CATIA V5 EasyToolbar

2 CATIA V5 EasyToolbar Solutions

3 Fast Installation and Usag

4 Demo Code Description

5 Software Subscription Instructions

6 How to publish your plugin

1. Why We Developed CATIA V5 EasyToolbar

1.1 Common Secondary Development Methods for CATIA V5

There are three primary secondary development methods for CATIA V5, each with distinct technical thresholds and application scenarios: CAA development, automation API development, and VBA macro development.

CAA development relies on the C++ programming language and requires Dassault Systèmes' exclusive RADE development environment, featuring an extremely high technical barrier and long learning curve, making it only suitable for ultra-complex customized development requirements.

Automation API development supports mainstream high-level languages such as C#, VB, and Python, and can be quickly mastered by developers with basic programming knowledge, balancing flexibility and practicality.

VBA macro development leverages CATIA's built-in macro recording function; users can record operations via the CATIA interface and modify the generated macros simply, enabling even entry-level CATIA users to get started with short-term learning.

With the exception of a tiny number of highly complex customized scenarios, automation API development can fully meet the vast majority of secondary development needs in engineering applications. It avoids the high complexity and configuration hurdles of CAA development, while also overcoming the limited functionality and poor scalability of VBA macro development. Overall, automation API development holds significant advantages in multiple critical dimensions, including talent recruitment, labor costs, learning cycles, development efficiency, and the commercialization of developed solutions.

1. Why We Developed CATIA V5 EasyToolbar

1.2 Core Pain Points of CATIA Automation API Development

Despite the prominent advantages of automation API development, it faces unavoidable technical bottlenecks in practical application that severely compromise user experience and development practicality.

First, as an out-of-process development method, automation API development cannot directly create custom toolbars or dropdown menus within the CATIA native interface, failing to achieve seamless integration with CATIA’s built-in UI.

Second, it has critical limitations in multi-instance compatibility: the standard interfaces GetObject(, "CATIA.Application") or Marshal.GetActiveObject("CATIA.Application") can only connect to the first opened CATIA instance, and cannot identify or access subsequent CATIA processes launched on the same machine.

Third, deployment restrictions apply: all automation API development outputs can only be compiled as standalone EXE files, which run outside the CATIA process and pop up separate operation windows, disrupting the unified and continuous operating experience of native CATIA workflows.

1. Why We Developed CATIA V5 EasyToolbar

1.3 Birth of CATIA V5 EasyToolbar

To resolve the core pain points of CATIA automation API development and break through these technical bottlenecks, CATIA V5 EasyToolbar was developed as a targeted solution.Initially, this tool was an internal utility customized to meet the engineering and R&D needs of our company, aimed at improving the efficiency of in-house engineers in CATIA secondary development and optimizing their operating experience.

After long-term verification in practical engineering projects and iterative functional optimization, it has been refined into a mature and universal professional tool. We have now officially launched it to global CATIA secondary development practitioners and engineering users, hoping to help more peers overcome technical dilemmas, lower development thresholds, and efficiently complete various customized CATIA development tasks.

CONTENTS

1 Why We Developed CATIA V5 EasyToolbar

2 CATIA V5 EasyToolbar Solutions

3 Fast Installation and Usage

4 Demo Code Description

5 Software Subscription Instructions

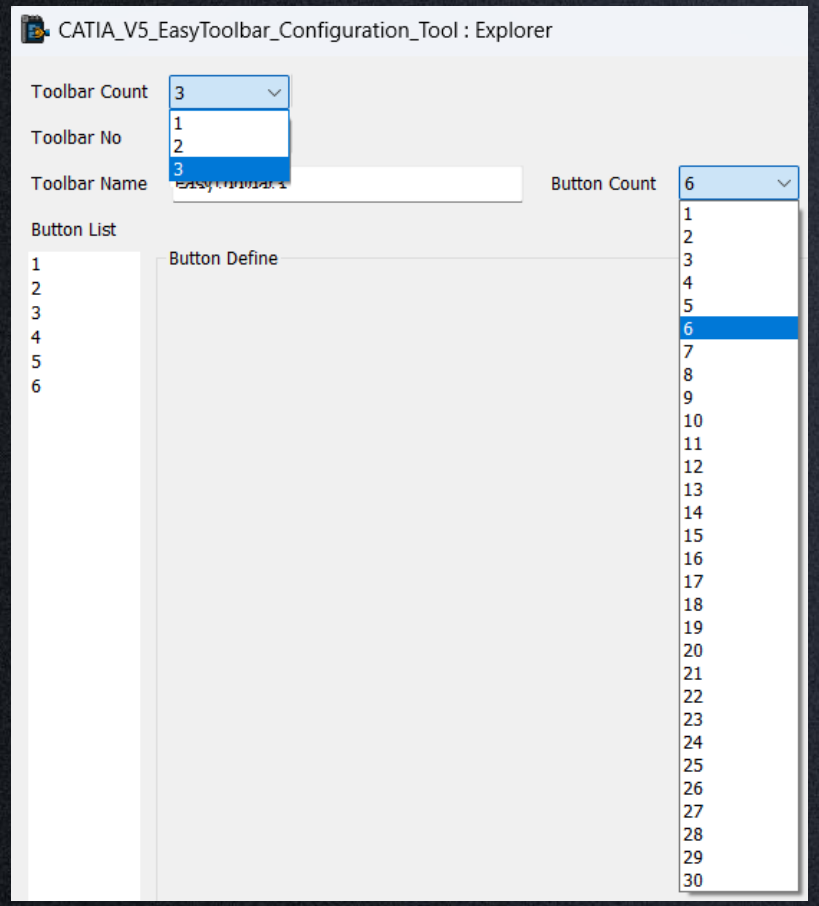
6 How to publish your plugin

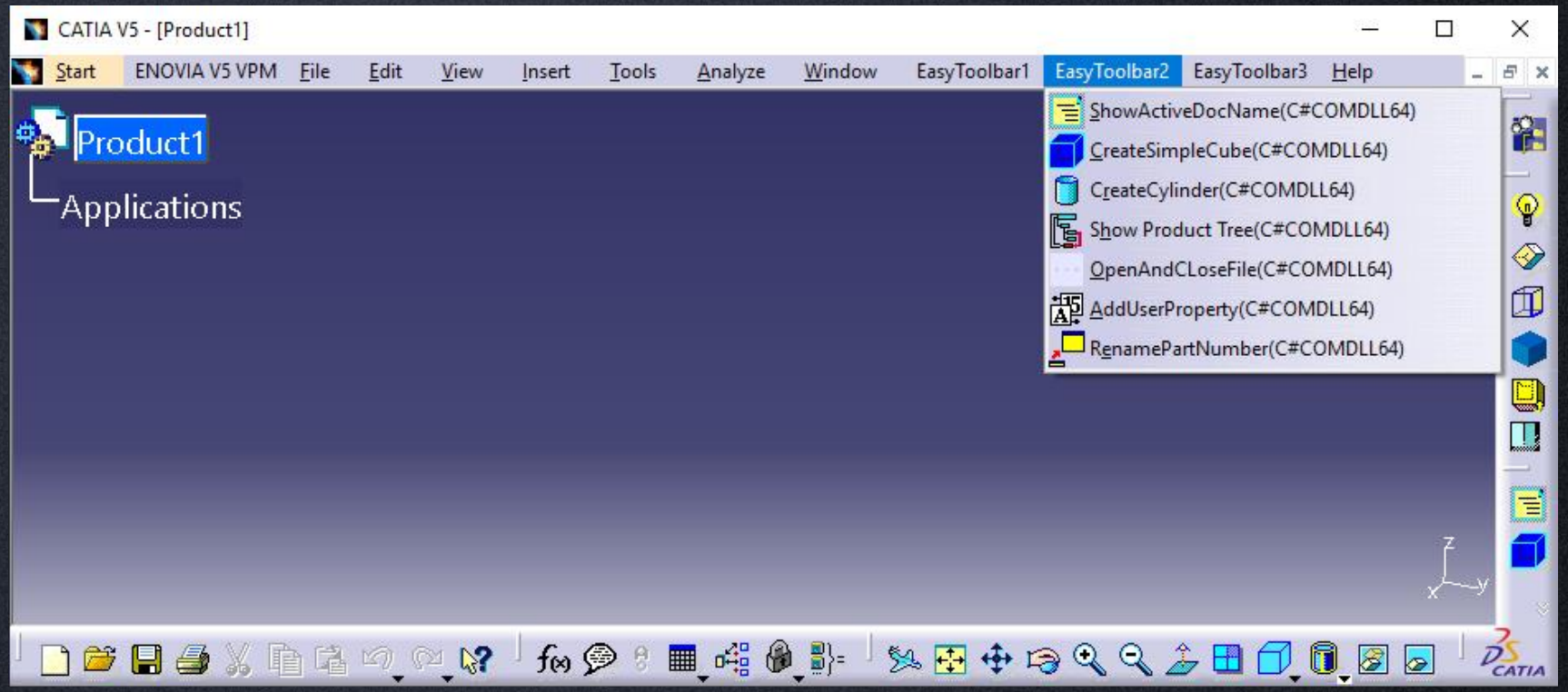
2. CATIA V5 EasyToolbar Solutions

2.1 Embedded Toolbars/Menus Configuration

Provides 3 configurable CATIA-embedded toolbars and menus, supporting CATIA V5 R18 to R36 (2026).64-bit CATIA only, Windows-only.

1-3 toolbars can be configured, with 1-30 buttons per toolbar. Users can customize the multi-language environment independently. It may also support R37 and newer versions, with specific adaptations to be made based on Dassault's updates.





As 32-bit versions of CATIA are rarely used nowadays, we have chosen not to support them.

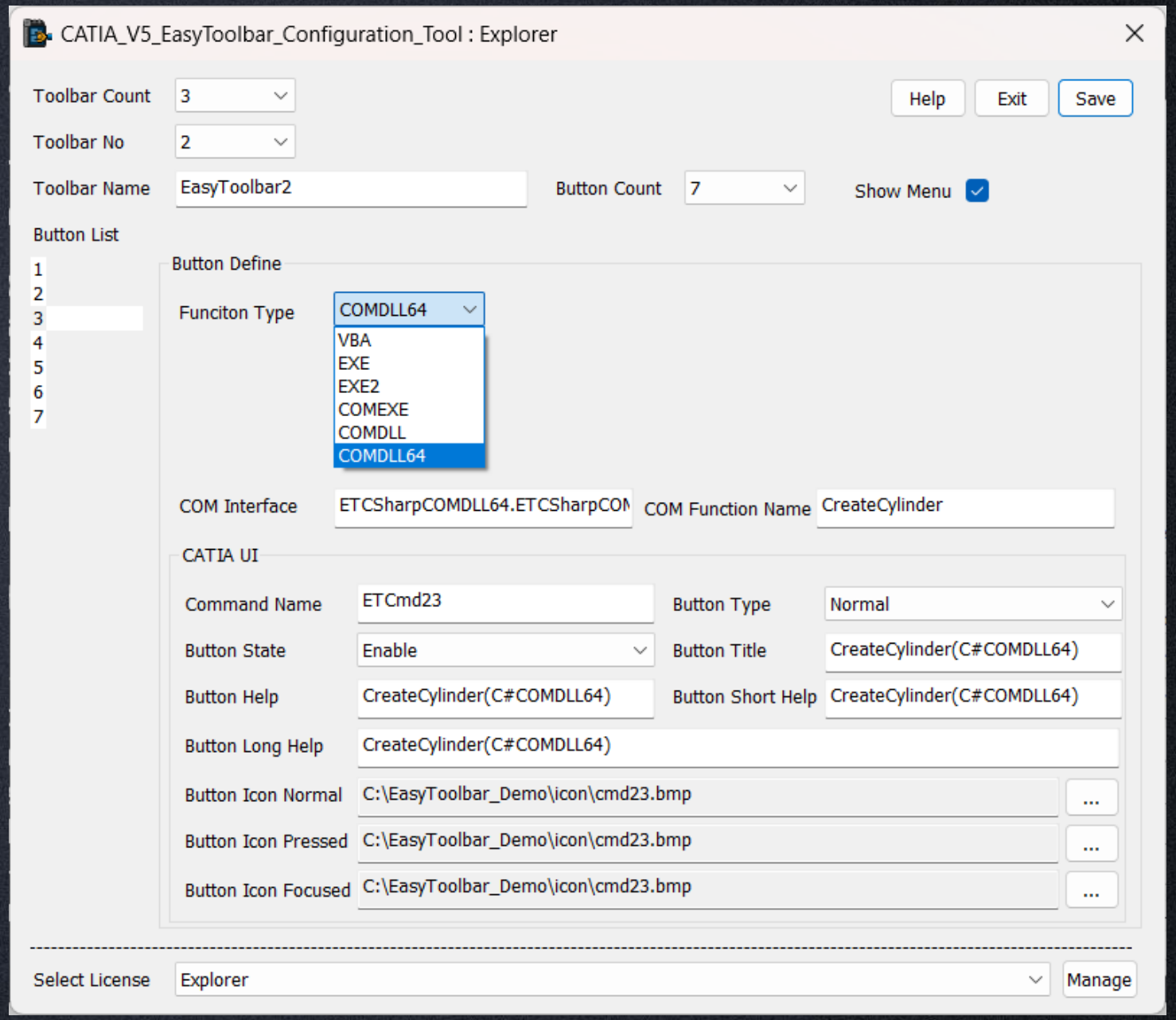
CATIA V6 (3DE) features a more modern development model, while EasyToolbar currently targets only CATIA V5.

2. CATIA V5 EasyToolbar Solutions

2.2 Toolbar Button Calling Methods

Supports 6 types of toolbar button calling methods: VBA, EXE, EXE2, COMEXE, COMDLL, COMDLL64. Button names, help information, images, etc., are configurable, and demo source code is provided for all these calling methods.

Supported development languages are VBA, VB, C# and VB.Net; Python and Java are not supported (because their support for COM interfaces is insufficient, we have tried many times without success, maybe you can). Since VB6 is too outdated and can only compile 32-bit programs, it is recommended to use C#+VBA. For C++ developers, while this tool can be used via types such as COMDLL64, it is recommended to use CAA for development — CATIA_V5_EasyToolbar is not designed for C++ users.



2. CATIA V5 EasyToolbar Solutions

2.3 Detailed Description

2.3.1 CATIA instances can be passed to the calling methods of EXE2, COMEXE, COMDLL, and COMDLL64, enabling the acquisition of the correct CATIA instance when multiple CATIA instances are opened.

VBA can always obtain the correct CATIA instance, while EXE cannot pass CATIA instances; please use EXE2 if needed.

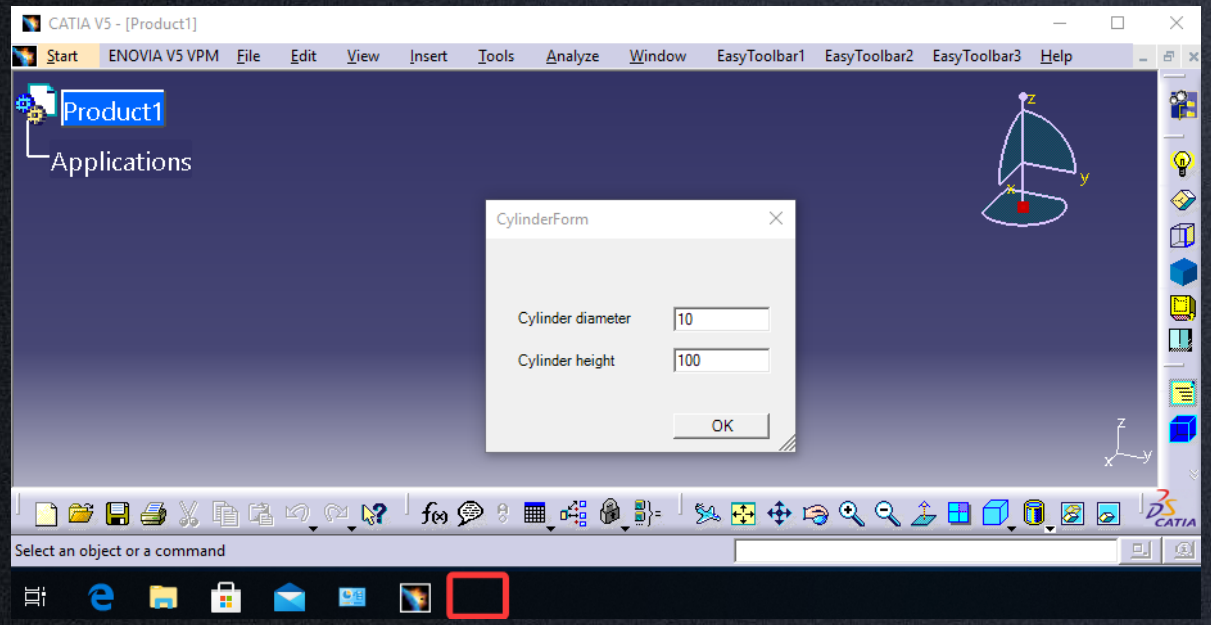
```
[Guid("6B29FC40-CA47-1067-B31D-00DD010662DA")]
[ProgId("ETCSharpCOMDLL64.ETCSharpCOMDLL64Class")]
[ClassInterface(ClassInterfaceType.None)]
[ComVisible(true)]

16 references
public class ETCSharpCOMDLL64Class : IETCSharpCOMDLL64
{
    7 references
    public ETCSharpCOMDLL64Class() { }
    0 references
    ~ETCSharpCOMDLL64Class() { }
    2 references
    public void ShowActiveDocName(ref object CATIA)
    {
        if (CATIA == null)
        {
            MessageBox.Show("C# COMDLL64 ShowActiveDocName: Input CATIA parameter is null");
            return;
        }

        try
        {
            //CATIA = (INFITF.Application)Marshal.GetActiveObject("catia.Application");
            INFITF.Application catia = (INFITF.Application)CATIA;
            MessageBox.Show("C# COMDLL64 ShowActiveDocName \r\n\r\n " + catia.ActiveDocument.FullName);
        }
```

2. CATIA V5 EasyToolbar Solutions

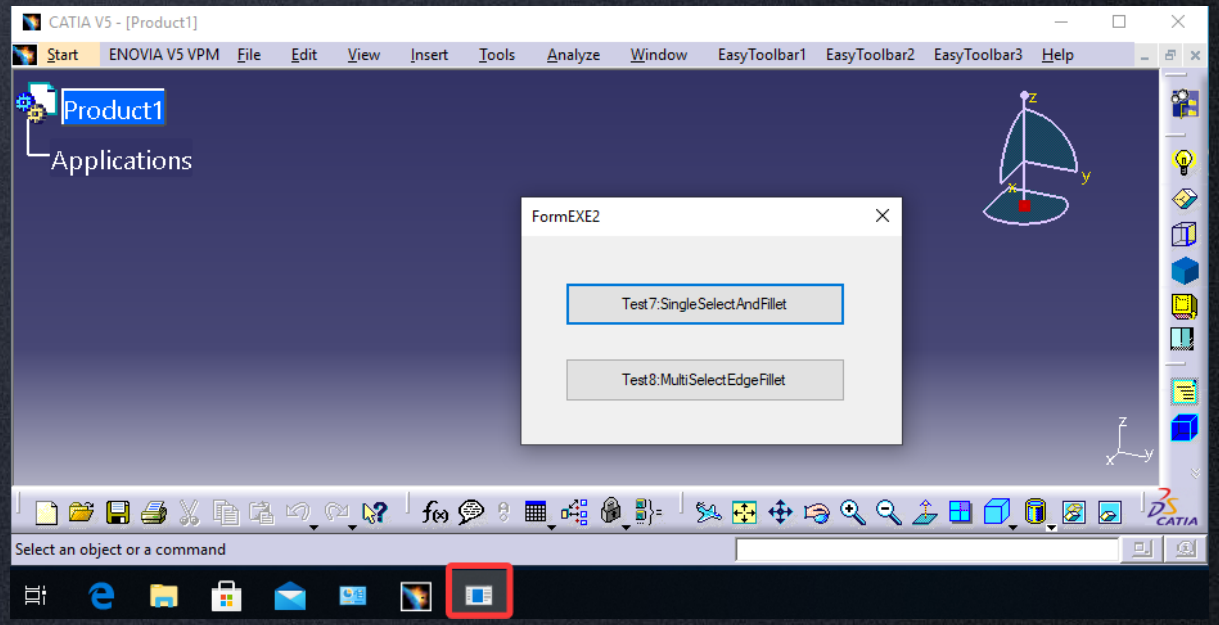
2.3 Detailed Description

2.3.2 The form interfaces called via VBA, COMDLL, and COMDLL64 pop up embedded in the CATIA environment. Those called via COMEXE, EXE, and EXE2 do not; they run in a separate process.

COMDLL64



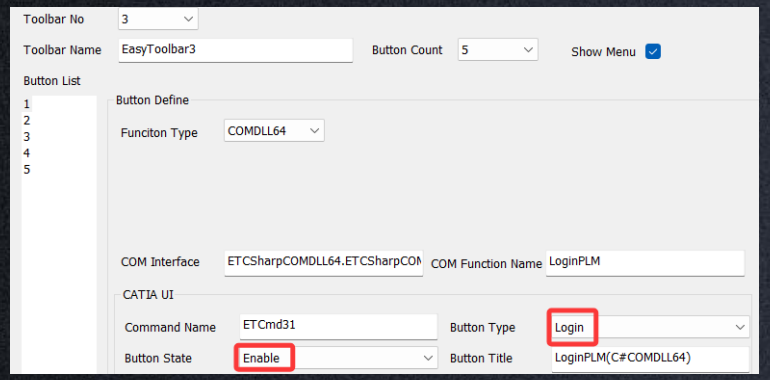
EXE2

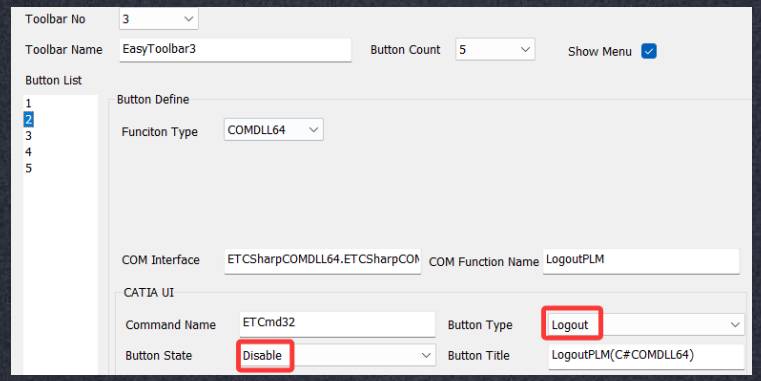


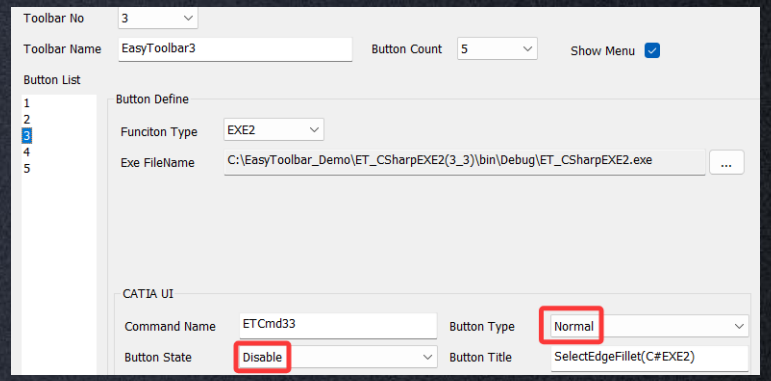
2. CATIA V5 EasyToolbar Solutions

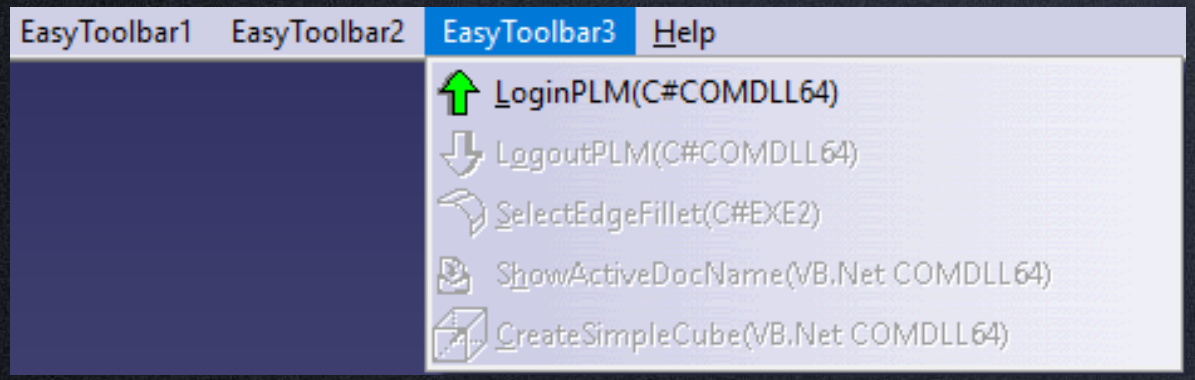
2.3 Detailed Description

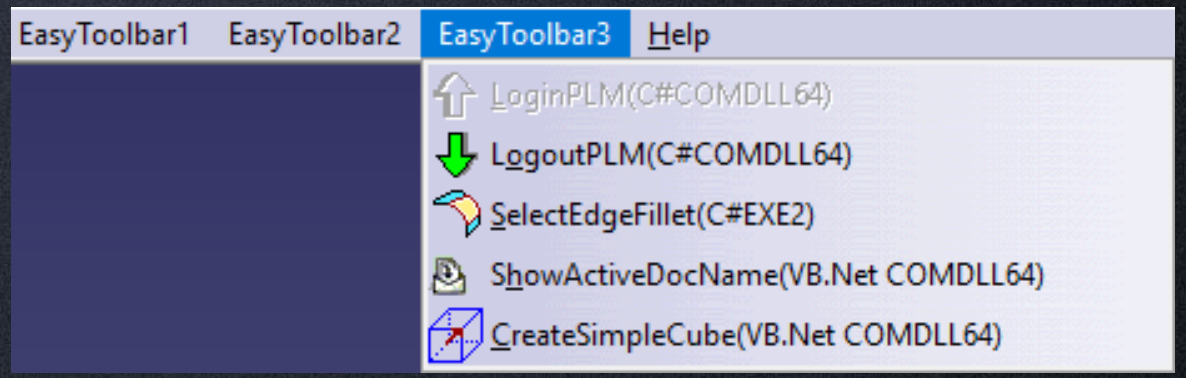
2.3.3 Toolbar buttons can enable or disable other buttons in the Login and Logout mode, facilitating the development of plugins for systems such as PLM. See the configuration of Toolbar No. 3 in the Demo.











2. CATIA V5 EasyToolbar Solutions

2.3 Detailed Description

2.3.4 Provides C# installation source code for CATIA V5 plugins for reference, including simultaneous installation and configuration under multiple versions, and automatic creation of shortcuts on the desktop. Please modify or rewrite it according to your own needs; this code is for demonstration purposes only. See the source code of CATIA_V5_EasyToolbar_Setup.

```
namespace CATIA_V5_EasyToolbar_Setup
{
    3 references
    public partial class Form_CATIA_V5_EasyToolbar_Setup : Form
    {
        private List<CATIAInfo> _catiaInfos = new List<CATIAInfo>();
        private VSCompatibilityResult compatibility;
        1 reference
        public Form_CATIA_V5_EasyToolbar_Setup()...
        1 reference
        private void InitListView()...
        1 reference
        private void Form_CATIA_V5_EasyToolbar_Setup_Load(object sender, EventArgs e)...
        1 reference
        private void LoadCATIAInfo()...
        1 reference
        private void buttonSetup_Click(object sender, EventArgs e)...
        1 reference
        private void InstallReg(string easytoolbarSetupPath)...
        1 reference
        private bool RegAsmComDll(string dllPath, bool is64bit = true,bool isRegister = true)...
        1 reference
        private bool Regsvr32ComDll(string dllPath, bool is64Bit, bool isRegister)...
        4 references
        private void CopyFolder(string sourceDirPath, string targetDirPath, bool overwriteExisting = true, bool
        2 references
        private int ExtractTrailingNumbers(string input)...
        1 reference
        private void CreateShortcutWithArguments(string shortcutPath, string arguments,string CATIAInstallPath)
```

CONTENTS

1 Why We Developed CATIA V5 EasyToolbar

2 CATIA V5 EasyToolbar Solutions

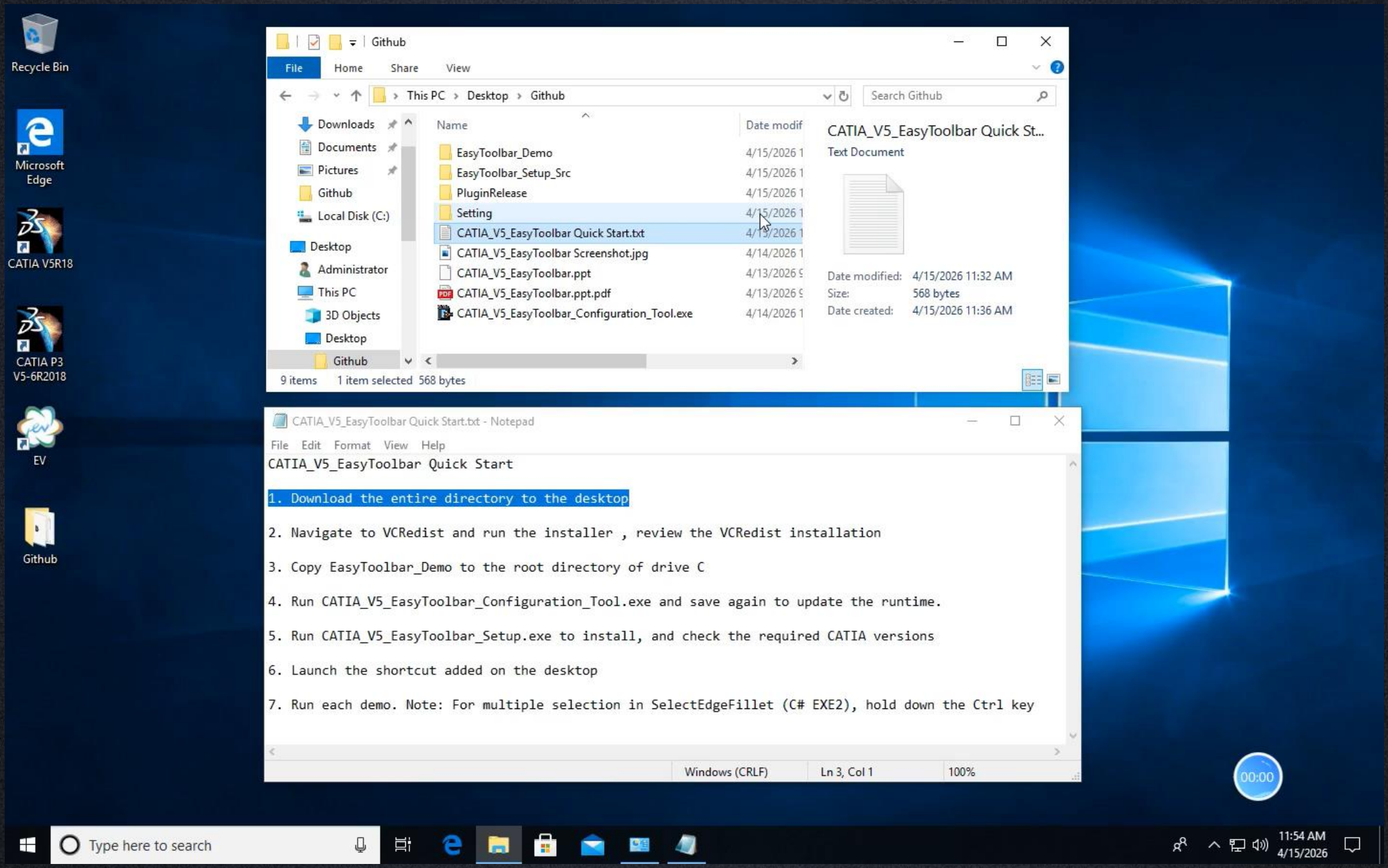
3 Fast Installation and Usage

4 Demo Code Description

5 Software Subscription Instructions

6 How to publish your plugin

3. Fast Installation and Usage



3. Fast Installation and Usage

CATIA_V5_EasyToolbar Quick Start

1. Download the entire directory to the desktop

2. Navigate to VCRedist and run the installer , review the VCRedist installation

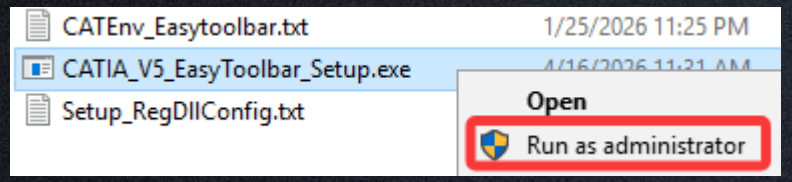
3. Copy EasyToolbar_Demo to the root directory of drive C

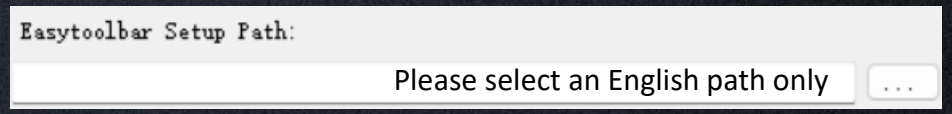
4. Run CATIA_V5_EasyToolbar_Configuration_Tool.exe and save again to update the runtime.

5. Run CATIA_V5_EasyToolbar_Setup.exe as administrator to install, and check the required CATIA versions.

6. Launch the shortcut added on the desktop

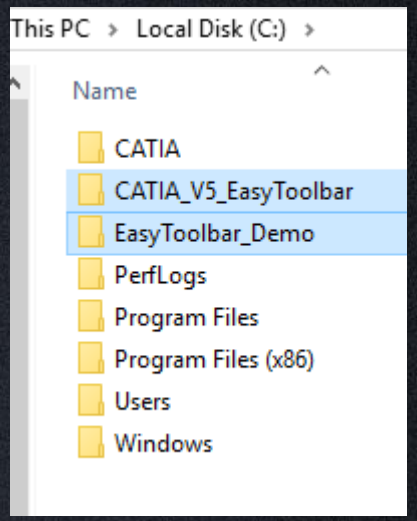
7. **Run each demo.** Note: For multiple selection in SelectEdgeFillet (C# EXE2), hold down the Ctrl key





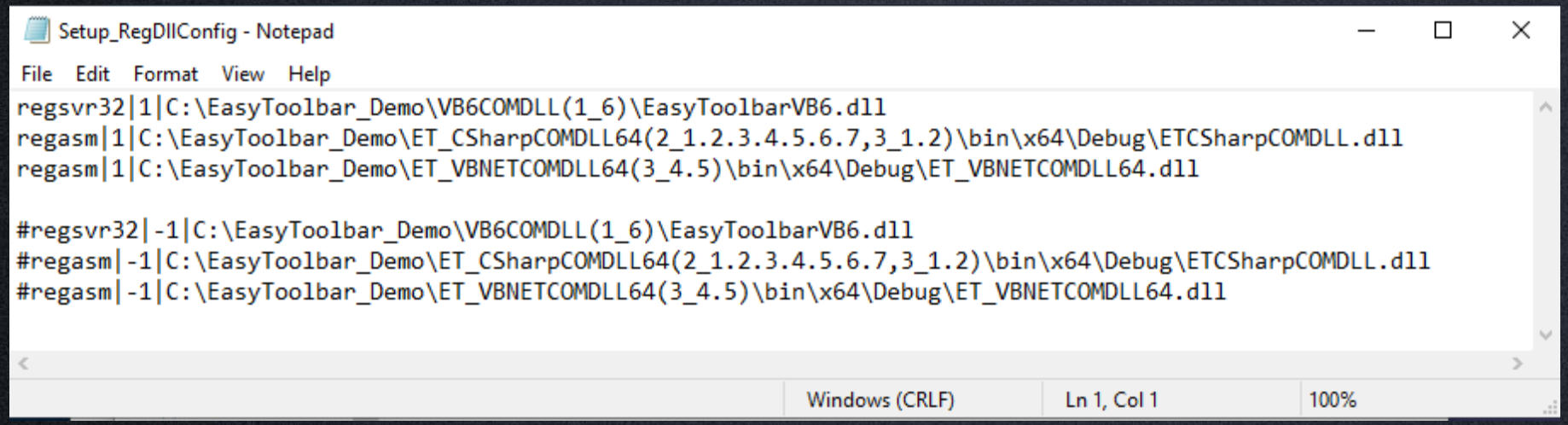
3. Fast Installation and Usage

After following the default steps in the video, two folders will be created on the C drive: EasyToolbar_Demo and CATIA_V5_EasyToolbar.



The folder C:\CATIA_V5_EasyToolbar is selected by CATIA_V5_EasyToolbar_Setup.exe.

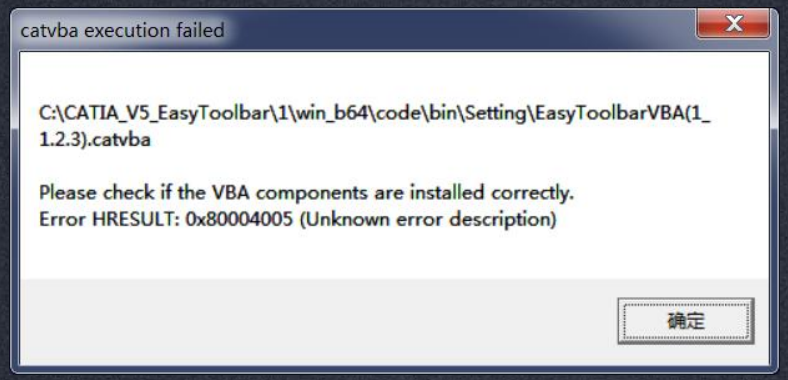
If you do not want to install the C:\EasyToolbar_Demo folder on the C drive, modify the relevant paths in Setup_RegDllConfig.txt (located in the same directory as CATIA_V5_EasyToolbar_Setup.exe), then run CATIA_V5_EasyToolbar_Configuration_Tool.exe to reassign new paths for CATVBA, EXE and ICON files.



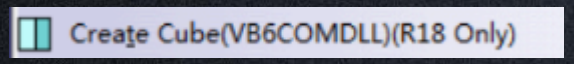
3. Fast Installation and Usage

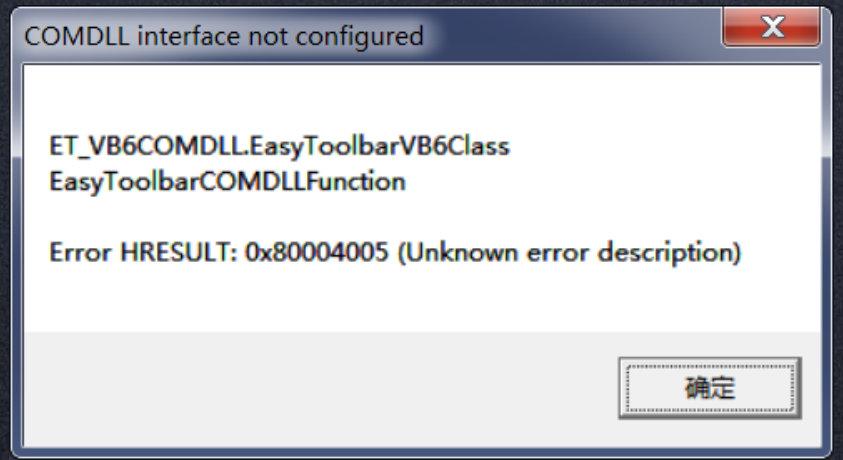
If CATIA_V5_EasyToolbar_Setup.exe fails to run, it may be because .NET Framework 4.5.1 or a later version is not installed on your computer.

If you encounter this error while running CATVBA, your CATIA VBA components are not properly installed.



If VB6COMDLL fails to run, please note that it supports only R18.





CONTENTS

1 Why We Developed CATIA V5 EasyToolbar

2 CATIA V5 EasyToolbar Solutions

3 Fast Installation and Usage

4 Demo Code Description

5 Software Subscription Instructions

6 How to publish your plugin

4. Demo Code Description

All source code used by the three toolbars in the Demo is located in the EasyToolbar_Demo directory, and the icon folder inside is used for storing icon files.

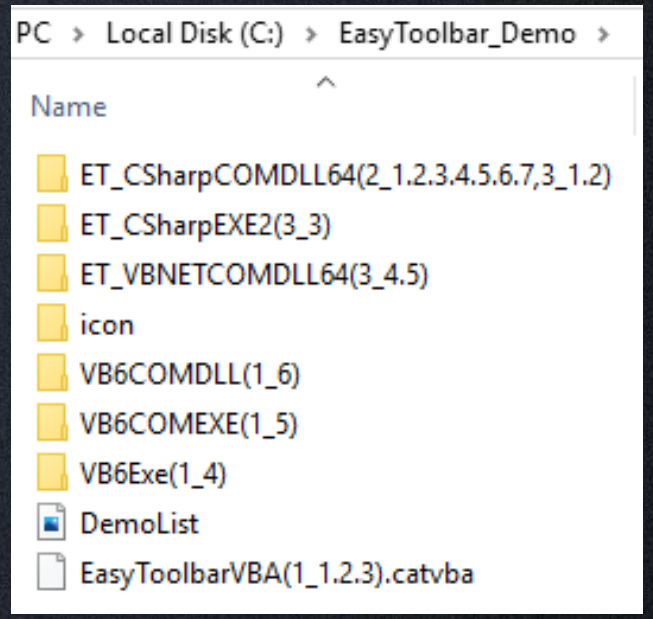
These source codes are free of any license restrictions, and much of the code was generated by AI and then adjusted.

CATVBA files can be directly opened and edited via CATIA's built-in macro editor.

The VB6 project requires Visual Basic 6.0 to be installed to open and edit.

The C# and VB.NET projects were developed using Visual Studio 2022 and target .NET Framework 4.5.1.

No	Toolbar No	Button No	Type	Code Category	Code Name	Description
1	1	1	VBA	Macro	EasyToolbarVBA(1_1.2.3).catvba	MsgBox : Product Name
2	1	2	VBA	Macro	EasyToolbarVBA(1_1.2.3).catvba	Create Cube(catvba)
3	1	3	VBA	Macro	EasyToolbarVBA(1_1.2.3).catvba	Recorded macro code：Create Box
4	1	4	EXE	VB6	VB6Exe(1_4)	Create Cube(VB6EXE)
5	1	5	COMEX	VB6	VB6COMEXE(1_5)	Create Cube(VB6COMEXE)
6	1	6	COMDLL	VB6	VB6COMDLL(1_6)	Create Cube(VB6COMDLL)
7	2	1	COMDLL64	C#	ET_CSharpCOMDLL64(2_1.2.3.4.5.6.7,3_1.2)	ShowActiveDocName
8	2	2	COMDLL64	C#	ET_CSharpCOMDLL64(2_1.2.3.4.5.6.7,3_1.2)	Create Cube
9	2	3	COMDLL64	C#	ET_CSharpCOMDLL64(2_1.2.3.4.5.6.7,3_1.2)	CreateCylinder (Input parameters)
10	2	4	COMDLL64	C#	ET_CSharpCOMDLL64(2_1.2.3.4.5.6.7,3_1.2)	Modal Dialog(Show Product Tree)
11	2	5	COMDLL64	C#	ET_CSharpCOMDLL64(2_1.2.3.4.5.6.7,3_1.2)	Open file / Close
12	2	6	COMDLL64	C#	ET_CSharpCOMDLL64(2_1.2.3.4.5.6.7,3_1.2)	AddUserProperty
13	2	7	COMDLL64	C#	ET_CSharpCOMDLL64(2_1.2.3.4.5.6.7,3_1.2)	RenamePartNumber
14	3	1	COMDLL64	C#	ET_CSharpCOMDLL64(2_1.2.3.4.5.6.7,3_1.2)	LoginPLM
15	3	2	COMDLL64	C#	ET_CSharpCOMDLL64(2_1.2.3.4.5.6.7,3_1.2)	LogoutPLM
16	3	3	EXE2	C#	ET_CSharpEXE2(3_3)	SingleSelectEdgeFillet MultiSelectEdgeFillet
17	3	4	COMDLL64	VB.Net	ET_VBNETCOMDLL64(3_4.5)	ShowCurDocName(VB.Net)
18	3	5	COMDLL64	VB.Net	ET_VBNETCOMDLL64(3_4.5)	Create Cube(VB.Net)



4. Demo Code Description

EasyToolbarVBA(1_1.2.3).catvba

Microsoft Visual Basic for Applications - C:\EasyToolbar_Demo\EasyToolbarVBA(1_1.2.3).catvba - [ModuleDemo1 (Code)]

File Edit View Insert Format Debug Run Tools Add-Ins Window Help

Ln 5, Col 1

Project - EasyToolbarVBA

EasyToolbarVBA (C:\EasyT

Modules

ModuleDemo1

ModuleDemo2

ModuleDemo3

(General)

```
Sub ShowCATDocumentName()
    MsgBox CATIA.ActiveDocument.Name
End Sub
```

Project - EasyToolbarVBA

EasyToolbarVBA (C:\EasyT

Modules

ModuleDemo1

ModuleDemo2

ModuleDemo3

(General)

```
'AI-generated code, only Step8 added..


' CATIA VBA Macro: Language-Agnostic Simple Cube (10x10x10mm)
' Final Fix: Avoid "IsGeometricalSet" (use PartBody = Bodies(1) - fixed rule for all CATIA V5)
' Compatibility: All CATIA V5 versions/languages (English/Chinese/French/German/R20-R32)
Sub CreateSimpleCube()
    ' Step 1: Initialize CATIA object (native to CATIA VBA, no GetObject)
    Dim oCATIA As Object
    Set oCATIA = CATIA

    ' Step 2: Create new Part document (fixed API parameter)
    Dim oPartDoc As Object
    Set oPartDoc = oCATIA.Documents.Add("Part")
```

Note: The code for Sub CreateBox() is a macro recorded in an English-language CATIA environment.It will throw an error when running in non-English CATIA environments. You may adjust it accordingly.

```
Dim body1 As Body
Set body1 = bodies1.Item("PartBody")
```

Project - EasyToolbarVBA

EasyToolbarVBA (C:\EasyT

Modules

ModuleDemo1

ModuleDemo2

ModuleDemo3

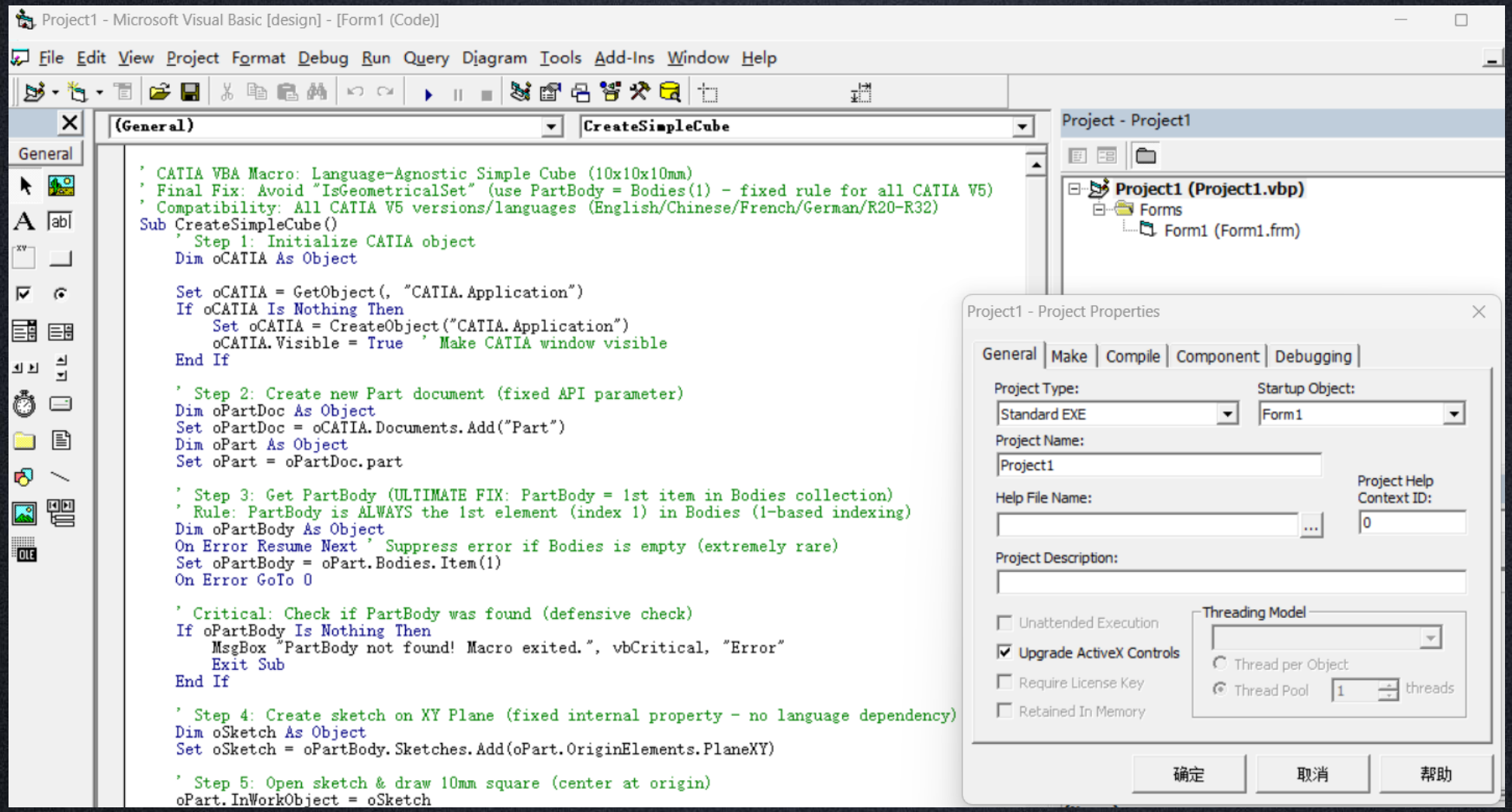
(General)

```
'CATIA macro-recorder code, only a MsgBox added.

Sub CreateBox()

MsgBox "This demo only runs in an English-language CATIA environment


Dim documents1 As Documents
Set documents1 = CATIA.Documents

Dim partDocument1 As PartDocument
Set partDocument1 = documents1.Add("Part")
```

4. Demo Code Description

VB6Exe(1_4)

This involves directly copying the macro code into a VB6 project.



4. Demo Code Description

VB6COMEXE(1_5)

Note that this is an ActiveX EXE.

ET_VB6COMEXE - Microsoft Visual Basic [design] - [EasyToolbarVB6Class (Code)]

File Edit View Project Format Debug Run Query Diagram Tools Add-Ins Window Help

(General) | (Declarations)

```
'Option Explicit

Public Function EasyToolbarCOMEXEFunction(CATIA As Object)

    'Set CATIA = CreateObject("CATIA.Application")

    If CATIA Is Nothing Then
        MsgBox "VB6 COMEXE Recived CATIA Parameter Is Nothing"
        Return
    End If

     MsgBox "VB6 COMEXE Show CATIA ActiveDocumentName : " + CATIA.ActiveDocument.Name, vbInformat:

     Call CreateSimpleCube(CATIA)

End Function


Sub CreateSimpleCube(oCATIA As Object)
    ' Step 1: Initialize CATIA object
    'Dim oCATIA As Object

    'Set oCATIA = GetObject(, "CATIA.Application")
    'If oCATIA Is Nothing Then
    ''     Set oCATIA = CreateObject("CATIA.Application")
    '     oCATIA.Visible = True  ' Make CATIA window visible
    'End If

    ' Step 2: Create new Part document (fixed API parameter)
    Dim oPartDoc As Object
    Set oPartDoc = oCATIA.Documents.Add("Part")
    Dim oPart As Object
    Set oPart = oPartDoc.part

    ' Step 3: Get PartBody (ULTIMATE FIX: PartBody = 1st item in Bodies collection)
    ' Rule: PartBody is ALWAYS the 1st element (index 1) in Bodies (1-based indexing)
    Dim oPartBody As Object
    On Error Resume Next ' Suppress error if Bodies is empty (extremely rare)
    Set oPartBody = oPart.Bodies.Item(1)
    On Error GoTo 0

    ' Critical: Check if PartBody was found (defensive check)
    If oPartBody Is Nothing Then
        MsgBox "PartBody not found! Macro exited.", vbCritical, "Error"
```

Project - ET_VB6COMEXE

- ET_VB6COMEXE (EasyToolbarVB6.vbp)
  - Class Modules
    - EasyToolbarVB6Class (EasyToolbarVB6Class.cls)

ET_VB6COMEXE - Project Properties

General | Make | Compile | Component | Debugging

Project Type: ActiveX EXE

Startup Object: (None)

Project Name: ET_VB6COMEXE

Help File Name:

Project Help Context ID: 0

Project Description:

Unattended Execution

Upgrade ActiveX Controls

Require License Key

Retained In Memory

Threading Model

Thread per Object

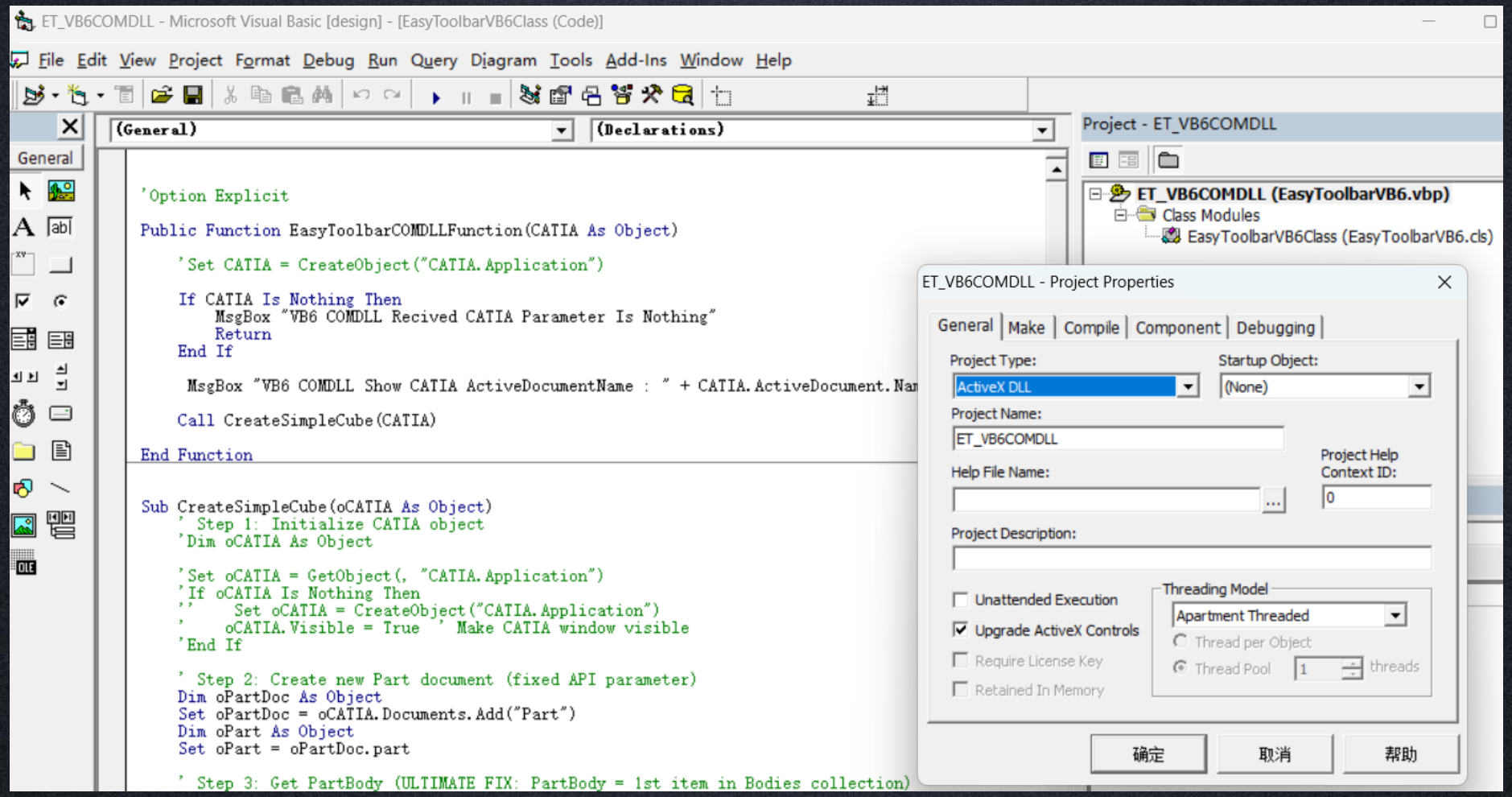
Thread Pool 1 threads

确定 取消 帮助

4. Demo Code Description

VB6COMDLL(1_6)

VB6 can only compile 32-bit DLLs. Even the mouse wheel doesn't work—it's really painful.
Note: COMDLL only supports older versions such as R18 and has limited compatibility with newer releases.
Please use COMDLL64 instead.



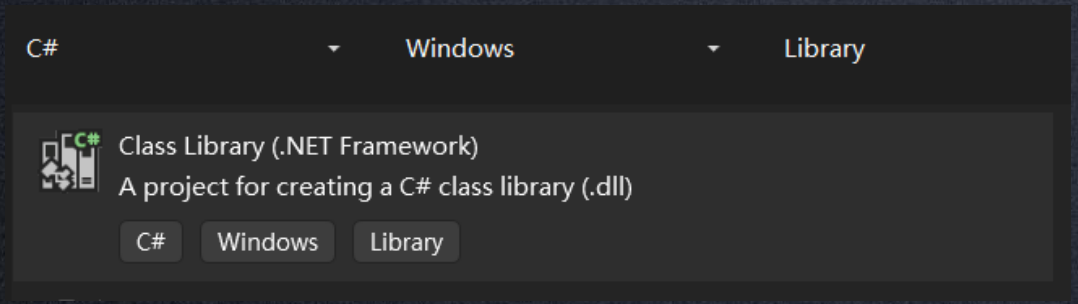
4. Demo Code Description

ET_CSharpCOMDLL64(2_1.2.3.4.5.6.7,3_1.2)

Please refer to the source code, as all details are contained within it.This project references the CATIA COM library and can only be compiled if CATIA is installed on your computer. Please make sure to compile for x64.

If you intend to modify and use this code directly, please update the namespace, Guid, ProgId, and class name to avoid conflicts with other developers.

If you want to create a new project, select the Library template based on .NET Framework. Note: .NET Framework fully supports IDispatch automation, whereas .NET Core/.NET 8+ only retains basic COM capabilities for cross-platform compatibility and has removed support for traditional Windows automation clients. You may still find a way to debug and make it work, and Microsoft may improve this support in the future.



4. Demo Code Description

ET_CSharpEXE2(3_3)

This EXE2 receives a CATIA instance from the command line, processes it, and then uses it.
Interactive functions such as SelectElement3 get stuck in COMDLL64, so they are executed in EXE2 instead.

```
namespace ET_CSharpEXE
{
    0 references
    internal static class Program
    {
        /// <summary>
        /// 应用程序的主入口点。
        /// </summary>
        [STAThread]
        0 references
        static void Main(string[] args)
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(fa

            FormEXE2 f = new FormEXE2();
            if (args.Length > 0)
            {
                string rotKey = args[0];
                f.rotKey = rotKey;
            }

            Application.Run(f);
        }
    }
```

```
private void MultiSelectEdgeFilletFunction(ref INFITF.Application catia)
{
    PartDocument partDoc = null;
    Part part = null;
    Selection sel = null;
    ShapeFactory shapeFactory = null;
    double Radius = 5.0;

    try
    {
        string extension = Path.GetExtension(catia.ActiveDocument.FullName).ToLower();
        if (extension != ".catpart")
        {
            MessageBox.Show("Please switch to the Part document to fillet");
            return;
        }

        partDoc = (PartDocument)catia.ActiveDocument;
        part = partDoc.Part;
        sel = partDoc.Selection;
        sel.Clear();

        object[] filter = new object[] { "Edge" };
        string status = sel.SelectElement3(filter, "Please select one or more edges (hold Ctrl to multi-select).",
                                           true, CATMultiSelectionMode.CATMultiSelTriggWhenUserValidatesSelection,

        if (status == "Normal")
        {
            int selectedCount = sel.Count;
```

4. Demo Code Description

ET_VBNETCOMDLL64(3_4.5)

VB.NET code is similar to C#. The core task is registering COM interfaces, and you must compile the project for x64 architecture.

This is a self-registering project.

Please run Visual Studio as an administrator.

Debug x64 Start

ET_VBNETCO...L64Class.vb

OMDLL64 IET_VBNETCOMDLL64 ShowActiveDocName

```vb
Imports System.Runtime.InteropServices
Imports System.Windows.Forms
Imports INFITF
Imports HybridShapeTypeLib
Imports MECMOD
Imports NavigatorTypeLib
Imports PARTITF
Imports ProductStructureTypeLib
Imports SPATypeLib


<Guid("9E5E5F0B-8D1A-4B2C-8E3D-4F5A6B7C8D9E")>
<InterfaceType(ComInterfaceType.InterfaceIsDual)>
3 references
Public Interface IET_VBNETCOMDLL64
    2 references
    Sub ShowActiveDocName(CATIA As Object)
    2 references
    Sub CreateSimpleCube(ByRef CATIA As Object)

End Interface

<Guid("1A2B3C4D-5E6F-7A8B-9C0D-1E2F3A4B5C6D")>
<ClassInterface(ClassInterfaceType.None)>
<ProgId("ET_VBNETCOMDLL64.ET_VBNETCOMDLL64Class")>
0 references
Public Class ET_VBNETCOMDLL64Class
    Implements IET_VBNETCOMDLL64

    2 references
    Public Sub ShowActiveDocName(CATIA As Object) Implements IET_VBNETCOMDLL64.ShowActiveDocName

        Dim catiaApp As INFITF.Application = CATIA
        MessageBox.Show(catiaApp.ActiveDocument.Name)

    End Sub
```

CONTENTS

1 Why We Developed CATIA V5 EasyToolbar

2 CATIA V5 EasyToolbar Solutions

3 Fast Installation and Usage

4 Demo Code Description

5 Software Subscription Instructions

6 How to publish your plugin

5. Software Subscription Instructions

CATIA_V5_EasyToolbar is a toolset developed and accumulated by our team over many years. It is available in two editions: Explorer and Subscription.Plugins released under the Explorer edition have a 2-week time limit, while plugins released under the Subscription edition have no time restrictions.

5.1 For Beginners

The provided source code enables beginners to get started quickly for introductory learning. By adding just one line of code to create a CATIA instance, the development process is identical to standard CATIA automation development. It can also be used as a tool for testing automation code.

5.2 For Students

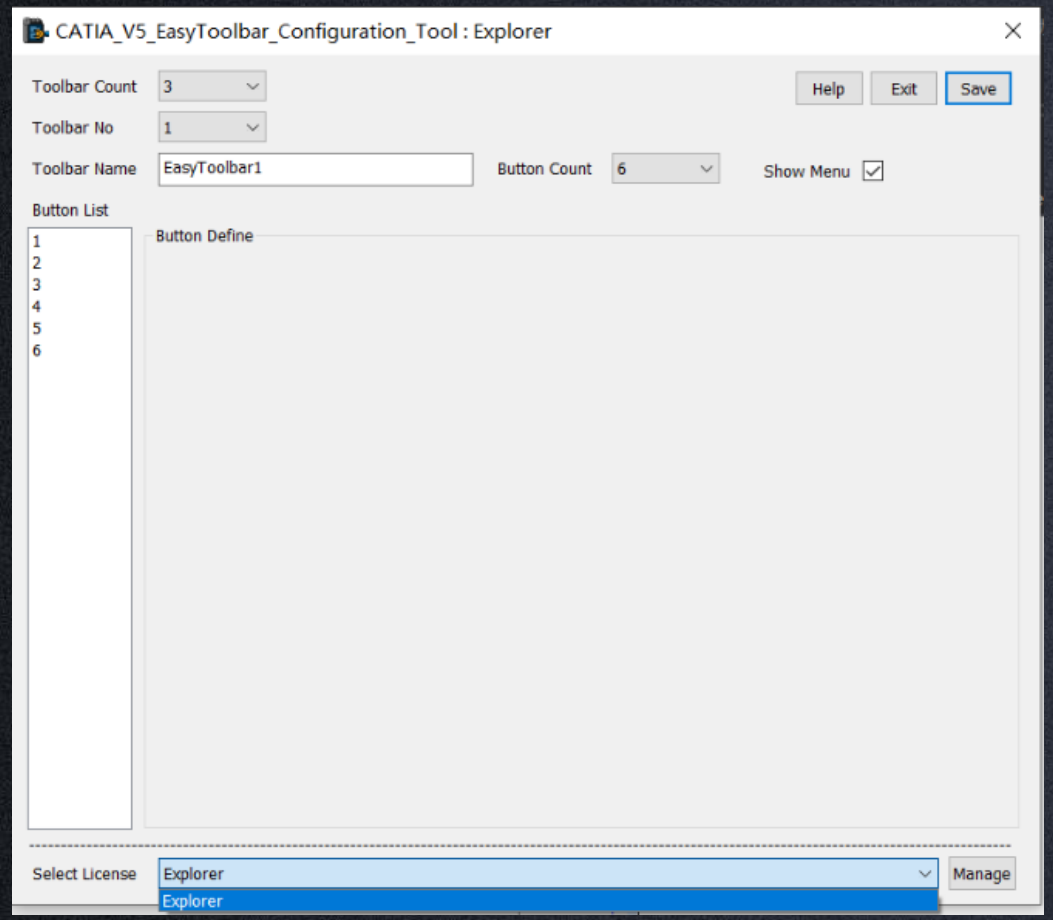
For students conducting basic research, working on projects, or writing papers, there is no need to distribute programs publicly. The usage period can be extended by another 2 weeks simply by re-saving the configuration, or continued use is possible by adjusting the plugin’s environment timestamp.

5.3 For Developers

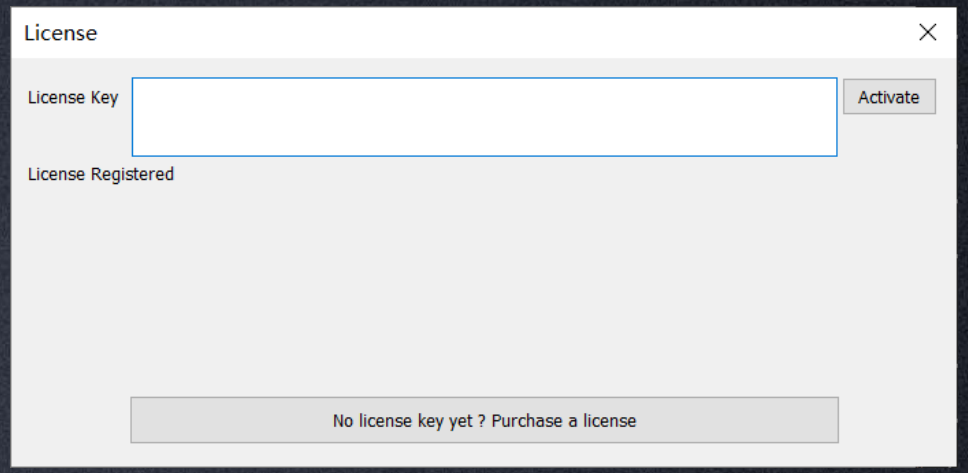
For developers, you may first develop and validate your projects in a test environment. When you need to distribute your plugin publicly, subscribe to CATIA_V5_EasyToolbar and configure the settings in the CATIA_V5_EasyToolbar_Configuration_Tool, then save the configuration.If you need to modify your code after distribution without changing the configuration via the tool, simply replace the CATVBA, EXE, or DLL files.

5. Software Subscription Instructions

Only Explorer is available when not subscribed.



Click Manage to open the activation window.

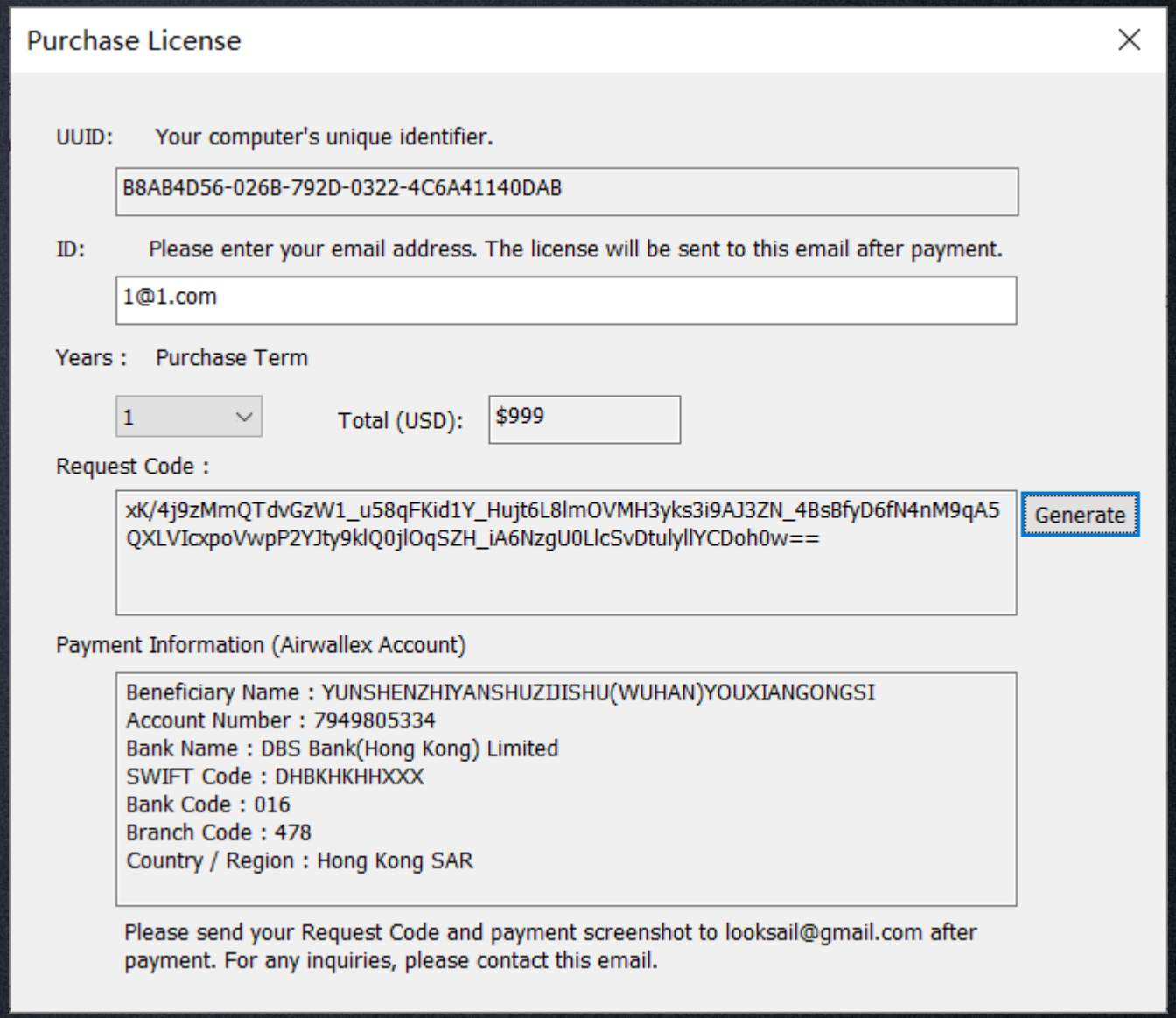


5. Software Subscription Instructions

Click the button to open the subscription form. Enter your email address as the ID, select the number of subscription years, and click the Generate button to create your Request Code.After completing payment, send the Request Code to looksail@gmail.com.

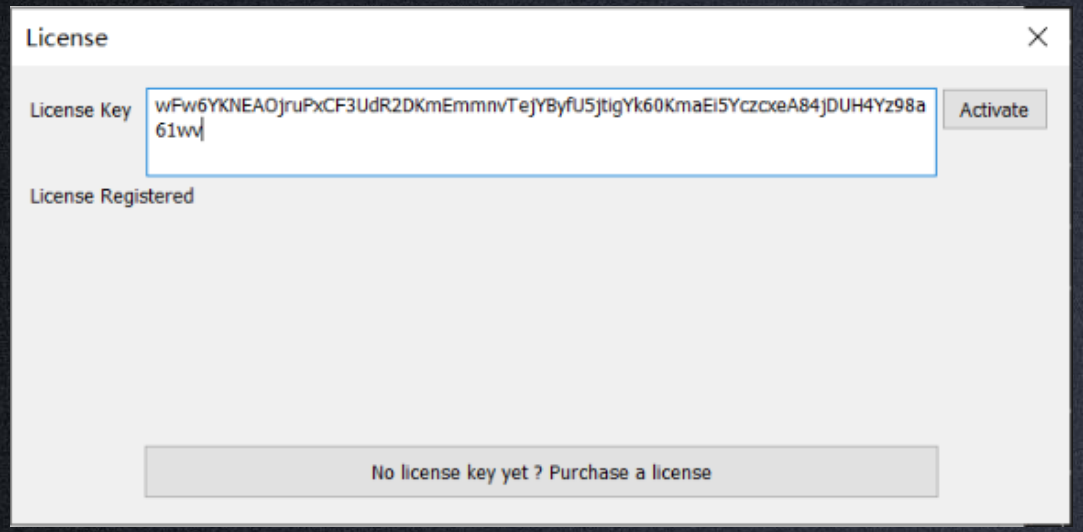
We will send the License Code to your registered email address within 1–3 business days after payment is received.

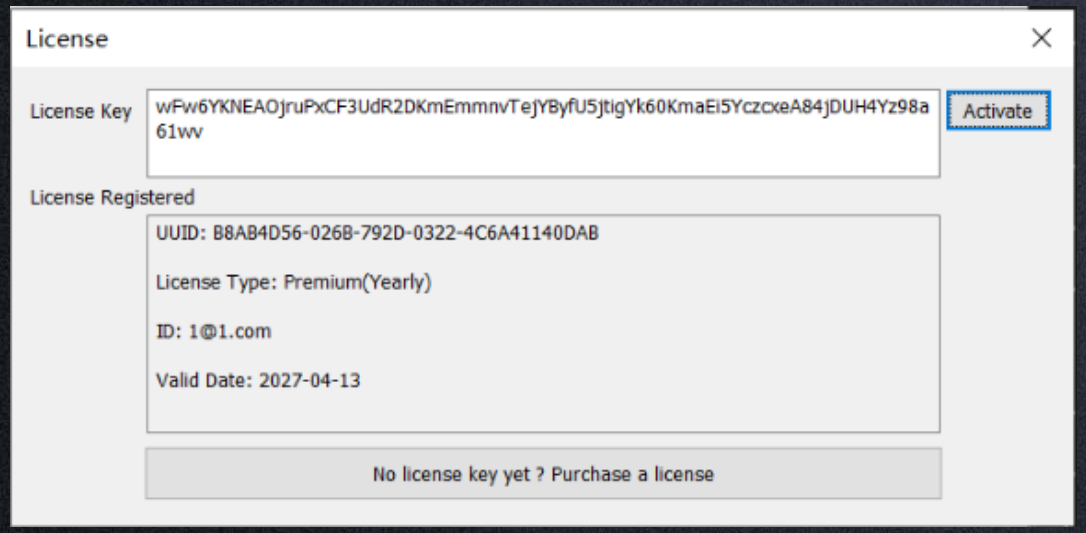
Thank you for your business and support.

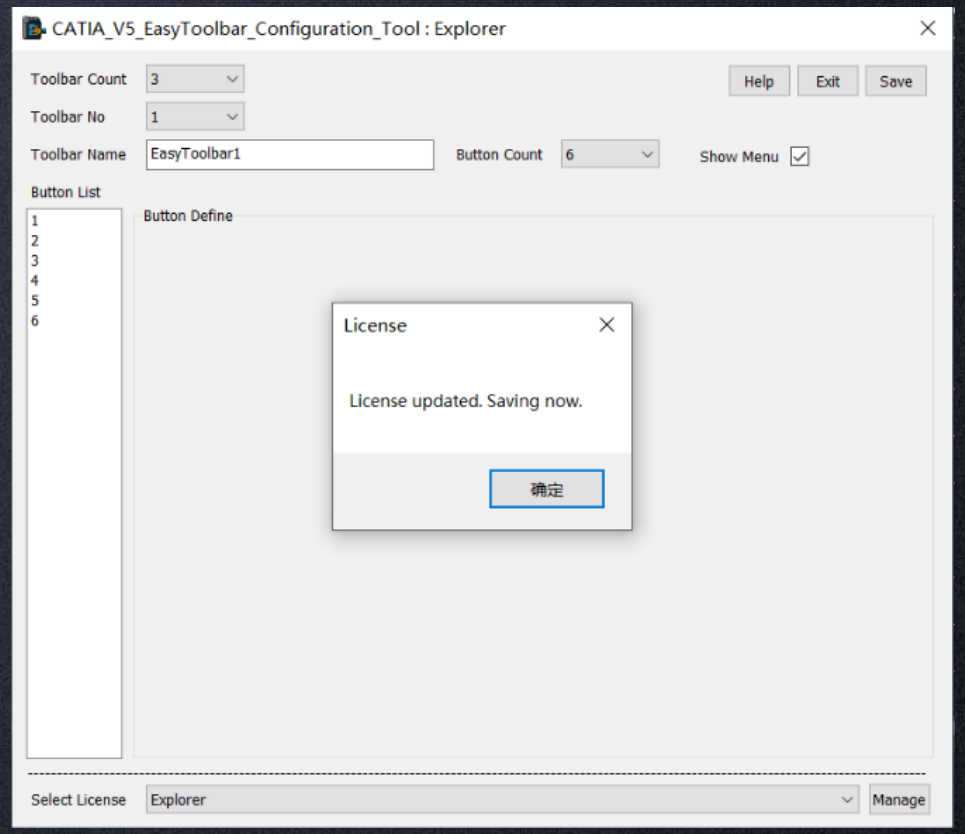


5. Software Subscription Instructions

Once you receive the License Code, copy it into the License Key field and click the Activate button.The subscription license will be saved automatically after you close the license window.

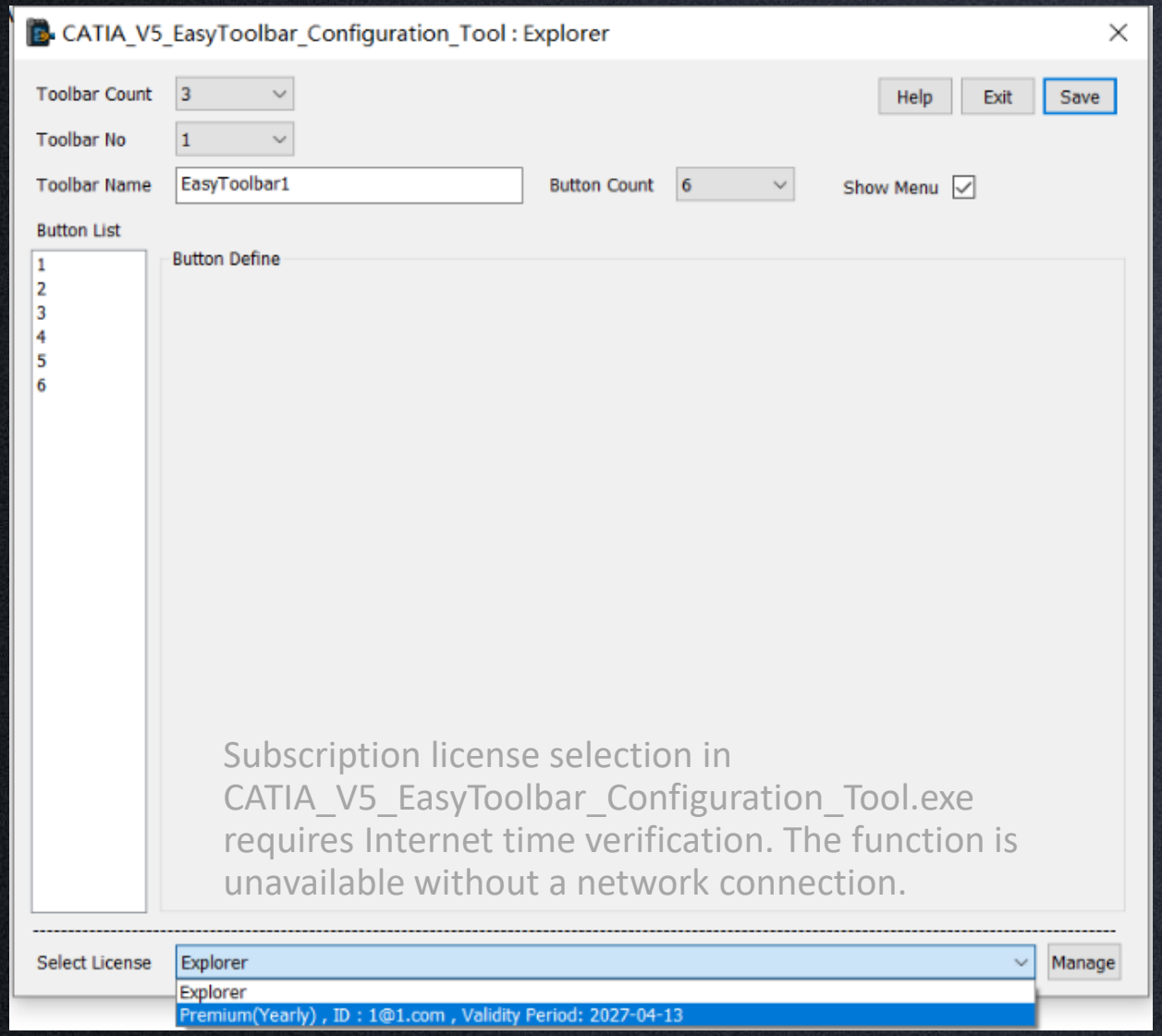


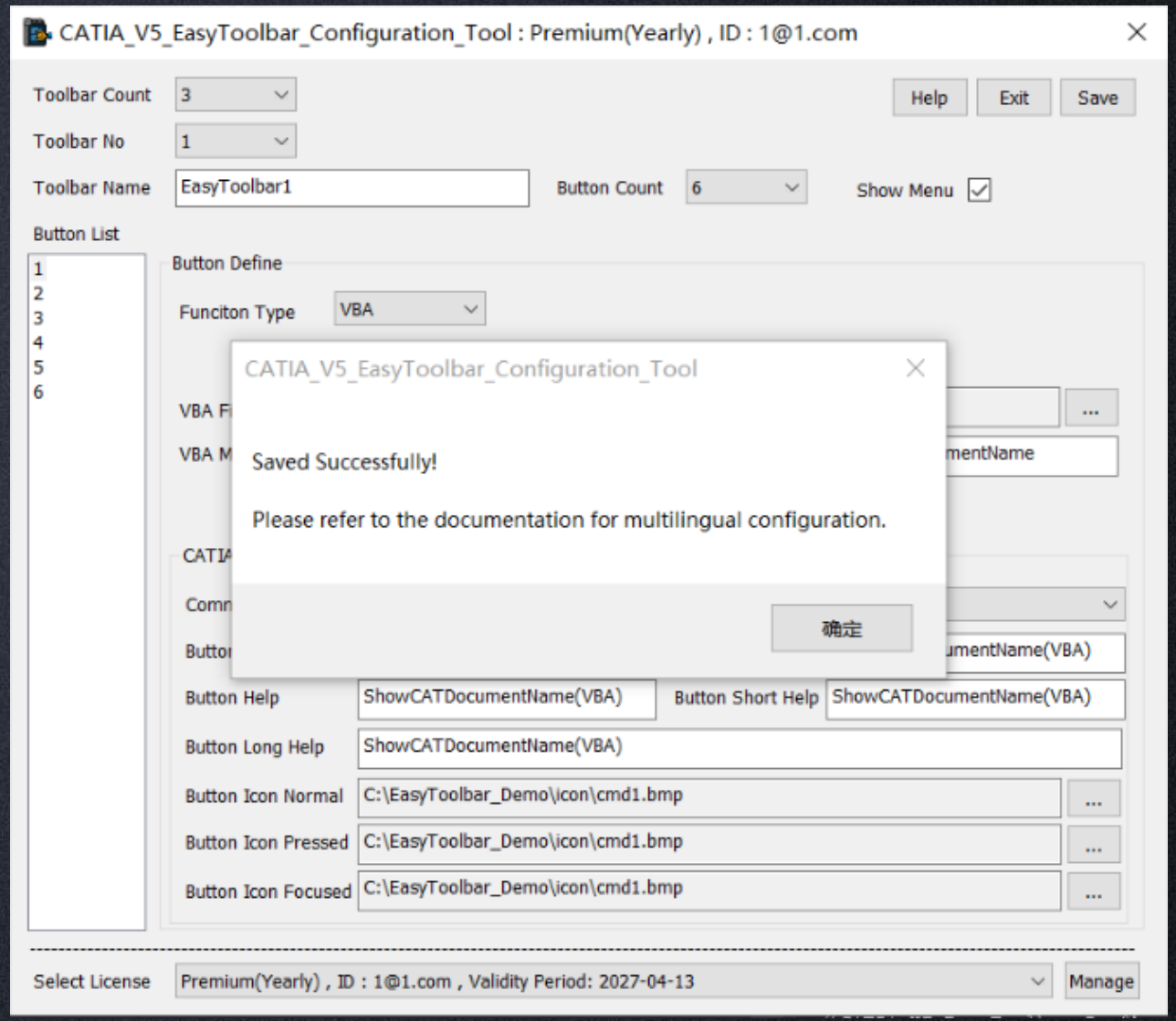




5. Software Subscription Instructions

You may then select your subscription license and click the Save button. The contents in the PluginRelease folder will be updated to the subscription version.



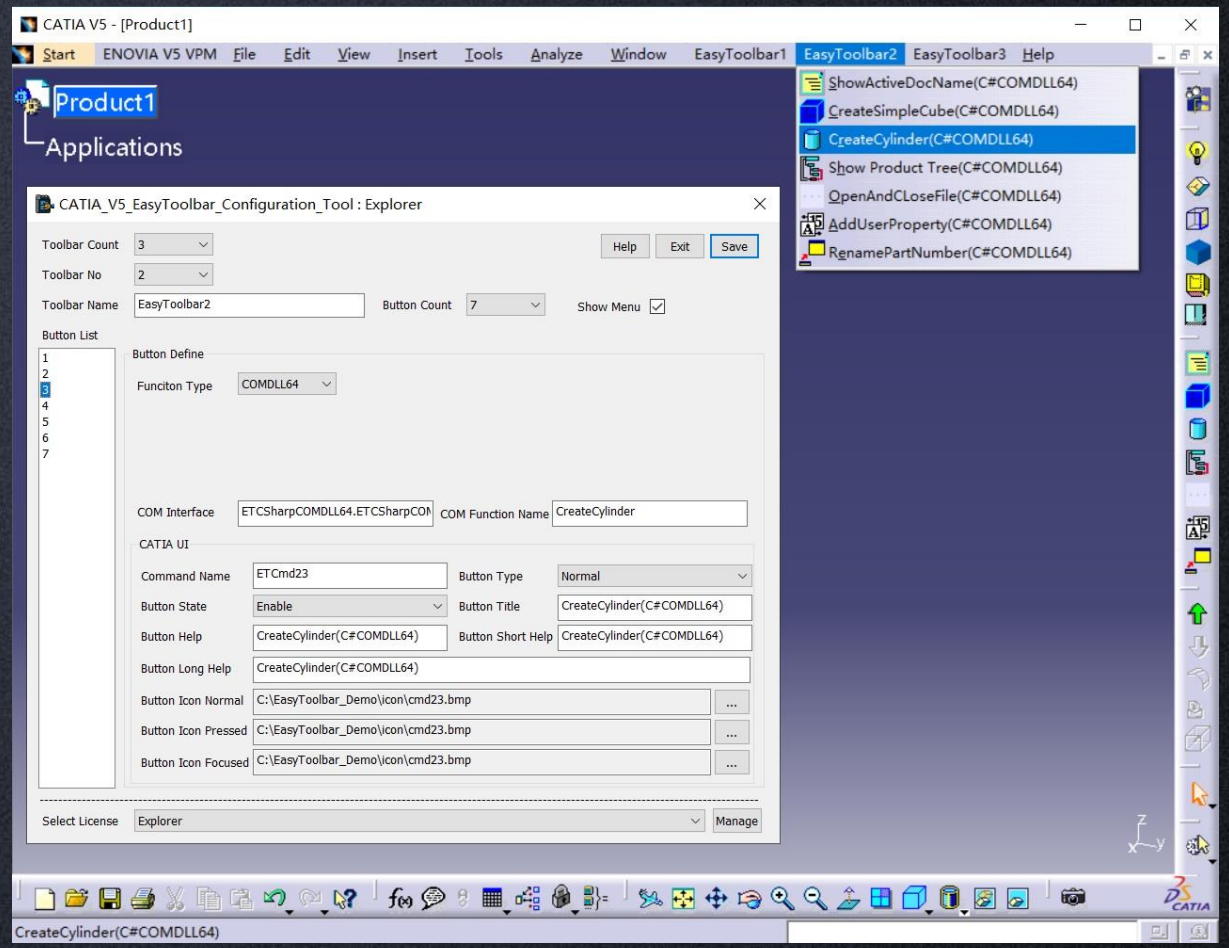


5. Software Subscription Instructions

Please first use the Explorer edition for code development, testing, publishing, and other tasks.

After fully verifying all functions and confirming stable operation with no issues, you may purchase a subscription before distributing plugins to your customers, thereby protecting your commercial interests.

Thank you for your business and support.



CONTENTS

1 Why We Developed CATIA V5 EasyToolbar

2 CATIA V5 EasyToolbar Solutions

3 Fast Installation and Usage

4 Demo Code Description

5 Software Subscription Instructions

6 How to publish your plugin

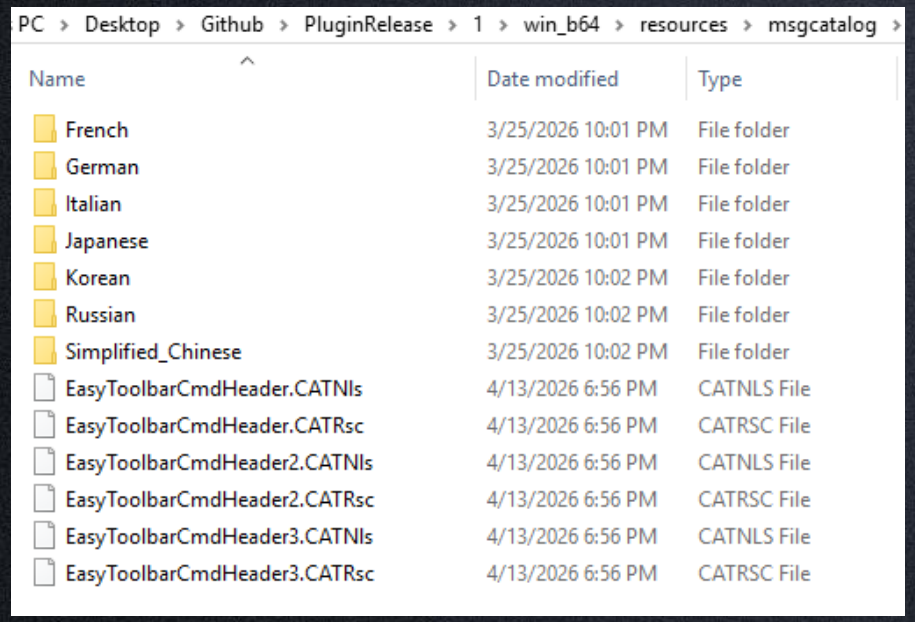
6. How to publish your plugin

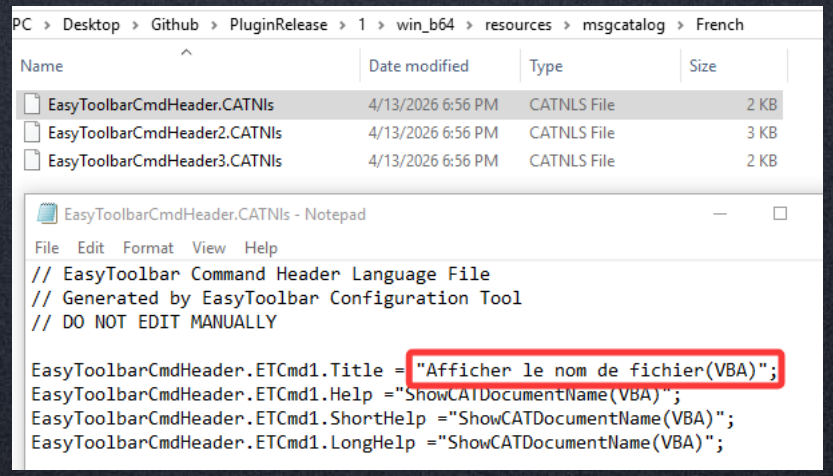
Multilingual Configuration:

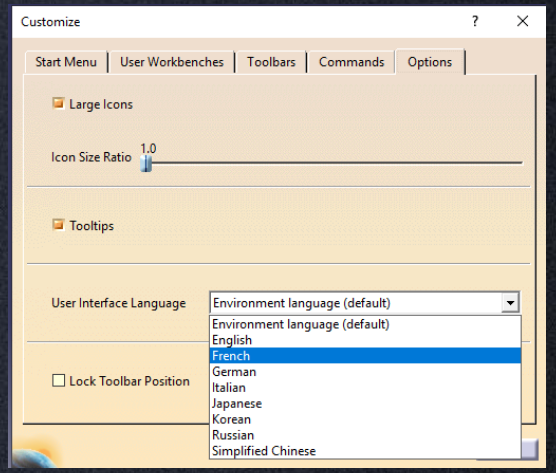
When you need to publish for multiple languages, copy the *.CATNls files from win_b64\resources\msgcatalog\ in Directory 1 to the target language directories under win_b64\resources\msgcatalog\, such as French, German, Simplified_Chinese, Italian, Japanese, Korean.

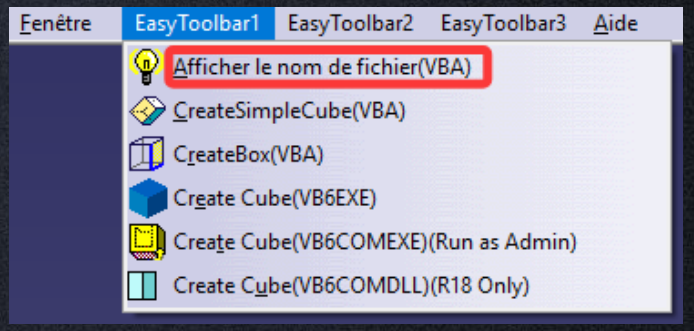
Then manually modify the text in the *.CATNls files within the target language directories to the desired language.This will allow the <Toolbar Button> labels in CATIA to display the corresponding text in different language environments.

Important: Ensure the modified files are saved in ANSI format.





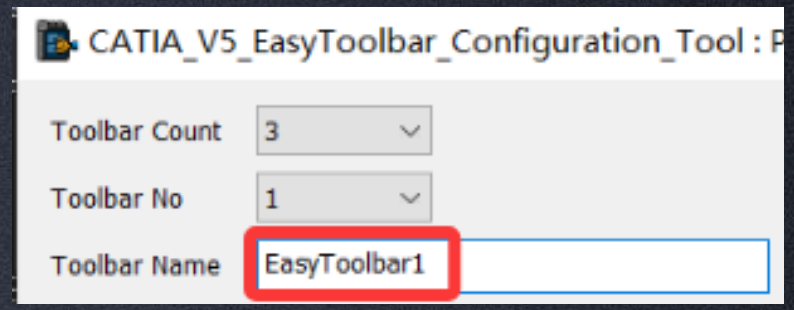




6. How to publish your plugin

Multilingual Configuration:

If the <Toolbar Name> contains localized text, the end-user system language must match the runtime language of CATIA_V5_EasyToolbar_Configuration_Tool.



After processing Directory 1, copy the modified target language directories (such as the French directory) from Directory 1 to Directory 2 and Directory 3.

6. How to publish your plugin

We provide the installation source code CATIA_V5_EasyToolbar_Setup as a demo for reference only.

However, some details have not been fully refined. For instance, if the same CATIA version is installed, uninstalled and reinstalled on the same device, the CATIA version information retrieved by the LoadCATIAInfo function will be incomplete and requires additional processing.

If you need to develop an uninstaller, please handle it on your own.

```
foreach (string subKeyName in baseKey.GetSubKeyNames())
{
    if (subKeyName.StartsWith("B"))
    {
        //The path may not only be directory 0, but also 1, 2, 3, etc. Further improvements are needed.
        string fullSubKeyPath = $"{subKeyName}\\0";
```

```
private void buttonSetup_Click(object sender, EventArgs e)
{
    //Setup Path
    string easytoolbarSetupPath = textBoxSetupPath.Text.Trim();
    if (string.IsNullOrEmpty(easytoolbarSetupPath) || !Directory.Exists(easytoolbarSetupPath))...

    easytoolbarSetupPath = Path.Combine(easytoolbarSetupPath, "CATIA_V5_EasyToolbar");
    if (!Directory.Exists(easytoolbarSetupPath))...

    //CATIA Version
    string appdataPath = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
    string catenvPath = Path.Combine(appdataPath, "DassaultSystemes", "CATEnv");
    if (!Directory.Exists(catenvPath))...
    string catenvFilename = Path.Combine(catenvPath, "CATIA_V5_EasyToolbar");
    List<CATIAInfo> selectedInfos = GetSelectedCATIAInfos();
    if (selectedInfos.Count <= 0)...
    int nVersion = -1;
    for (int i = 0; i < selectedInfos.Count; i++)...

    //Copy file
    string exeFullPath = Assembly.GetExecutingAssembly().Location;
    string exeDir = Path.GetDirectoryName(exeFullPath);
    if (string.IsNullOrEmpty(exeDir))
    {
        exeDir = Environment.CurrentDirectory;
    }

    CopyFolder(Path.Combine(exeDir, "1"), Path.Combine(easytoolbarSetupPath, "1"));
    CopyFolder(Path.Combine(exeDir, "2"), Path.Combine(easytoolbarSetupPath, "2"));
    CopyFolder(Path.Combine(exeDir, "3"), Path.Combine(easytoolbarSetupPath, "3"));

    //Write Settting File & Create Shortcut
    //C:\Users\Administrator\AppData\Roaming\DassaultSystemes\CATEnv
    for (int i = 0; i < selectedInfos.Count; i++)...

    //COMDLL COMDLL64 need regasm/regsvr32
    InstallReg(easytoolbarSetupPath);

    //Finish
    DialogResult result = MessageBox.Show(
        "Installed successfully. Do you want to exit the installer?",
        "Installation Complete",
```

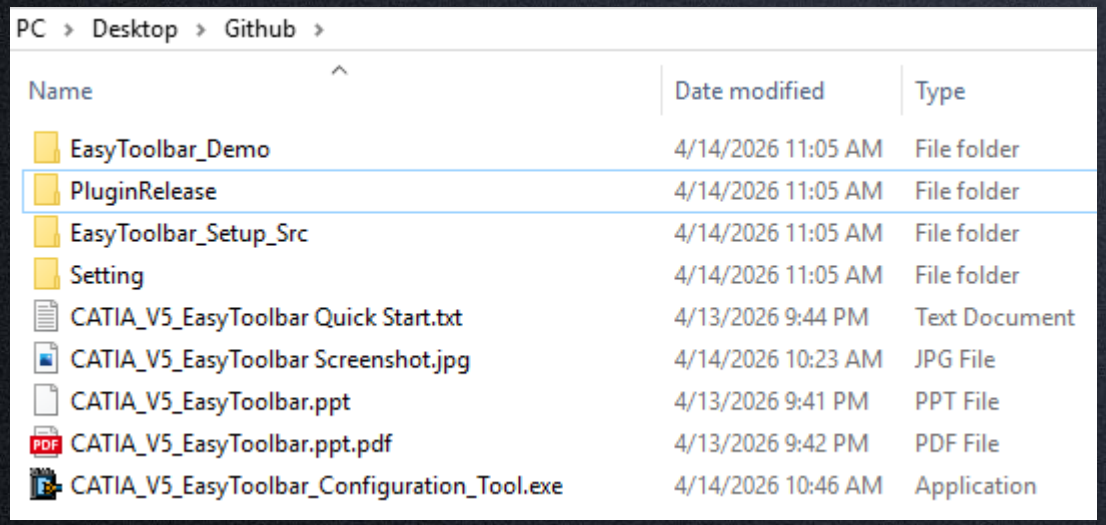
6. How to publish your plugin

The EXE and CATVBA files will be copied to the Setting directories 1, 2, and 3 during configuration, but the COMDLL and COMDLL64 files will not be copied automatically.

If you use CATIA_V5_EasyToolbar_Setup directly:

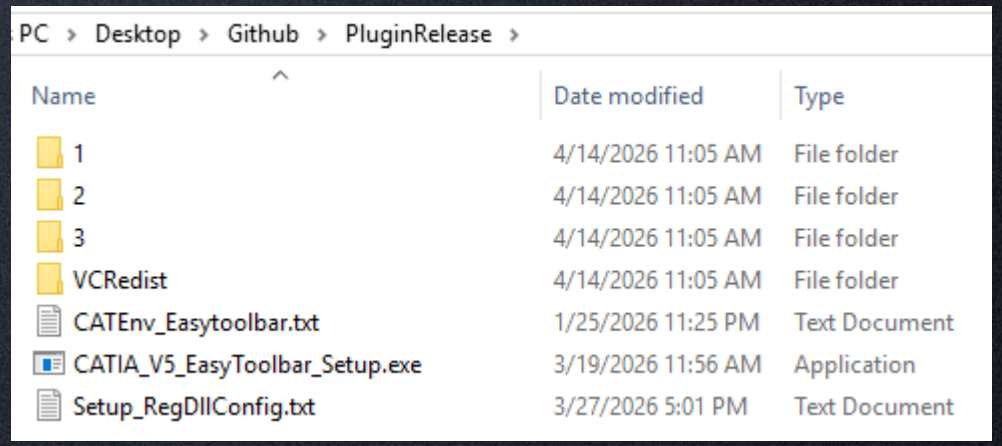
1. Modify the source code in EasyToolbar_Setup_Src to add file copy operations for your own COMDLL and COMDLL64 files , etc. Remember to check the dependencies of your DLLs.
2. Compile the project and copy the output to the PluginRelease folder.Update the COM registration information in Setup_RegDllConfig.txt.
3. Finally, distribute the PluginRelease folder to your users for installation.

For more advanced requirements, you can create a separate installer. Refer to the source code in EasyToolbar_Setup_Src for detailed operations.



PC › Desktop › Github ›

Name	Date modified	Type
EasyToolbar_Demo	4/14/2026 11:05 AM	File folder
PluginRelease	4/14/2026 11:05 AM	File folder
EasyToolbar_Setup_Src	4/14/2026 11:05 AM	File folder
Setting	4/14/2026 11:05 AM	File folder
CATIA_V5_EasyToolbar Quick Start.txt	4/13/2026 9:44 PM	Text Document
CATIA_V5_EasyToolbar Screenshot.jpg	4/14/2026 10:23 AM	JPG File
CATIA_V5_EasyToolbar.ppt	4/13/2026 9:41 PM	PPT File
CATIA_V5_EasyToolbar.ppt.pdf	4/13/2026 9:42 PM	PDF File
CATIA_V5_EasyToolbar_Configuration_Tool.exe	4/14/2026 10:46 AM	Application



PC › Desktop › Github › PluginRelease ›

Name	Date modified	Type
1	4/14/2026 11:05 AM	File folder
2	4/14/2026 11:05 AM	File folder
3	4/14/2026 11:05 AM	File folder
VCRedist	4/14/2026 11:05 AM	File folder
CATEnv_Easytoolbar.txt	1/25/2026 11:25 PM	Text Document
CATIA_V5_EasyToolbar_Setup.exe	3/19/2026 11:56 AM	Application
Setup_RegDllConfig.txt	3/27/2026 5:01 PM	Text Document

CATIA_V5_EasyToolbar

V2.01

Technical Documentation & User Manual

End

Support & Contact: looksail@gmail.com

Copyright © 2026 All Rights Reserved.